50 Java Interview Questions

For Freshers & Internship Candidates

Covers Core Java • OOP • Collections • Exception Handling • Multithreading • JDBC

SECTION 1: Core Java Basics

Q1. What is Java? What are its main features?

Java is a high-level, object-oriented, platform-independent programming language developed by Sun Microsystems (now Oracle). Key features: Object-Oriented, Platform Independent (Write Once Run Anywhere), Secure, Multithreaded, Robust, Portable, High Performance.

Q2. What is the difference between JDK, JRE, and JVM?

- JVM (Java Virtual Machine) – Executes bytecode; platform-specific.
- JRE (Java Runtime Environment) – JVM + libraries needed to run Java programs.
- JDK (Java Development Kit) – JRE + development tools like compiler (javac) and debugger.

Q3. What is the difference between == and .equals() in Java?

== compares object references (memory addresses), while .equals() compares the actual content/values of objects. For String comparisons, always use .equals().

```
String a = "hello";
String b = new String("hello");
System.out.println(a == b);       // false
System.out.println(a.equals(b));  // true
```

Q4. What are the 8 primitive data types in Java?

byte, short, int, long, float, double, char, boolean

Q5. What is the difference between Stack and Heap memory?

- Stack – Stores local variables and method calls. Memory is automatically freed when method ends.
- Heap – Stores objects and instance variables. Managed by Garbage Collector.

Q6. What is autoboxing and unboxing?

Autoboxing is the automatic conversion of a primitive type to its wrapper class (e.g., int → Integer). Unboxing is the reverse (Integer → int). Introduced in Java 5.

Q7. What is a static keyword in Java?

static means the member belongs to the class rather than any instance. Static variables/methods can be accessed without creating an object. Static blocks run once when the class is loaded.

Q8. What is the final keyword?

- final variable – Value cannot be changed (constant).
- final method – Cannot be overridden.
- final class – Cannot be subclassed (e.g., String class).

Q9. What is the difference between String, StringBuilder, and StringBuffer?

• String – Immutable. Every modification creates a new object.
• StringBuilder – Mutable, not thread-safe, faster.
• StringBuffer – Mutable, thread-safe (synchronized), slightly slower.

Q10. What is typecasting in Java?

Converting one data type to another. Widening (automatic, e.g. int→long) and Narrowing (manual, e.g. double→int) casting are the two types.

■ SECTION 2: Object-Oriented Programming (OOP)

Q11. What are the 4 pillars of OOP?

1. Encapsulation – Wrapping data and methods together; hiding internal details.
2. Inheritance – A class acquires properties of another class.
3. Polymorphism – One interface, many forms (overloading & overriding).
4. Abstraction – Hiding implementation, showing only essential features.

Q12. What is the difference between method overloading and overriding?

• Overloading – Same method name, different parameters (compile-time polymorphism).
• Overriding – Subclass provides a specific implementation of a superclass method (runtime polymorphism).

Q13. What is an abstract class vs interface?

• Abstract class – Can have abstract and non-abstract methods; supports constructors; single inheritance.
• Interface – All methods are abstract by default (Java 7); supports multiple inheritance; no constructors.
From Java 8, interfaces can have default and static methods.

Q14. What is a constructor? Types of constructors?

A constructor is a special method called when an object is created. Types:
• Default constructor – No parameters, provided by Java if none defined.
• Parameterized constructor – Takes arguments.
• Copy constructor – Copies values from another object.

Q15. What is 'this' keyword in Java?

this refers to the current object instance. Used to differentiate instance variables from local variables, call another constructor in the same class, or pass the current object as a parameter.

Q16. What is 'super' keyword?

super refers to the parent class. Used to call parent class constructor (super()), access parent class methods (super.method()), and access parent class variables.

Q17. What is encapsulation? How to achieve it?

Encapsulation is binding data (variables) and methods together and restricting direct access to data. Achieved by:
• Declaring variables as private.
• Providing public getter and setter methods.

Q18. What is polymorphism? Give an example.

Polymorphism means 'many forms'. Example of runtime polymorphism: A parent class reference variable can hold a child class object, and the overridden method of the child class is called at runtime.

```
Animal a = new Dog();
a.sound(); // calls Dog's sound() method
```

Q19. Can we override static methods?

No. Static methods belong to the class, not instances. What appears to be overriding static methods is actually method hiding, not true polymorphism.

Q20. What is the difference between an object and a class?

- Class – A blueprint or template defining properties and behaviors.
- Object – An instance of a class with actual values assigned.

❑ SECTION 3: Exception Handling

Q21. What is exception handling in Java?

Exception handling is a mechanism to handle runtime errors gracefully using try, catch, finally, throw, and throws keywords, so the normal flow of the program is maintained.

Q22. Difference between checked and unchecked exceptions?

- Checked exceptions – Checked at compile time (e.g., IOException, SQLException). Must be handled.
- Unchecked exceptions – Occur at runtime (e.g., NullPointerException, ArrayIndexOutOfBoundsException). Not mandatory to handle.

Q23. What is the difference between throw and throws?

- throw – Used to explicitly throw an exception inside a method.
- throws – Used in method signature to declare that the method might throw an exception.

Q24. What is the finally block?

The finally block always executes after try-catch, regardless of whether an exception occurred or not. Used for cleanup operations like closing database connections or file streams.

Q25. What is the difference between Error and Exception?

- Error – Serious problems that the application cannot recover from (e.g., OutOfMemoryError, StackOverflowError).
- Exception – Conditions that the application can catch and handle.

❑ SECTION 4: Collections Framework

Q26. What is the Java Collections Framework?

A unified architecture for storing and manipulating groups of objects. Key interfaces: List, Set, Queue, Map. Common classes: ArrayList, LinkedList, HashSet, TreeSet, HashMap, TreeMap.

Q27. Difference between ArrayList and LinkedList?

- ArrayList – Uses dynamic array; fast random access O(1); slow insert/delete in middle O(n).
- LinkedList – Uses doubly linked list; slow random access O(n); fast insert/delete O(1).

Q28. Difference between HashMap and HashTable?

- HashMap – Not synchronized (not thread-safe), allows one null key, faster.
- HashTable – Synchronized (thread-safe), does not allow null keys, slower.

Q29. Difference between HashSet and TreeSet?

- HashSet – Unordered, allows null, backed by HashMap, O(1) operations.
- TreeSet – Sorted in natural order, no null allowed, backed by TreeMap, O(log n) operations.

Q30. What is the difference between List, Set, and Map?

- List – Ordered, allows duplicates (ArrayList, LinkedList).
- Set – Unordered, does not allow duplicates (HashSet, TreeSet).
- Map – Stores key-value pairs, keys are unique (HashMap, TreeMap).

Q31. What is an Iterator?

An Iterator is an object used to traverse a Collection. It provides hasNext() to check for more elements, next() to get the next element, and remove() to remove elements during iteration.

Q32. What is the difference between Iterator and ListIterator?

- Iterator – Can traverse only forward; works with all collections.
- ListIterator – Can traverse both forward and backward; works only with List.

Q33. What is a ConcurrentModificationException?

Thrown when a collection is modified while iterating over it using an Iterator (except through the iterator's own remove method). Use Iterator.remove() or CopyOnWriteArrayList to avoid this.

■ SECTION 5: Multithreading & Concurrency

Q34. What is multithreading?

Multithreading is the ability of a CPU to execute multiple threads concurrently. In Java, it helps in better CPU utilization and building responsive applications.

Q35. Ways to create a thread in Java?

1. Extending Thread class and overriding run().
2. Implementing Runnable interface and passing to Thread constructor.
3. Using Callable and Future (for return values).
4. Using ExecutorService (thread pools).

Q36. What is synchronization?

Synchronization controls access to shared resources by multiple threads to prevent data inconsistency. The synchronized keyword ensures only one thread accesses a block/method at a time.

Q37. What is a deadlock?

Deadlock occurs when two or more threads are blocked forever, each waiting for the other to release a lock. Prevention: avoid nested locks, use lock ordering, use tryLock() with timeout.

Q38. What is the volatile keyword?

volatile ensures that a variable's value is always read from and written to main memory, not from thread-local cache. Useful for flags in multithreading but does not guarantee atomicity.

■ SECTION 6: Java 8+ Features

Q39. What are Lambda Expressions?

Lambda expressions provide a concise way to represent anonymous functions (functional interfaces). Syntax: (parameters) -> expression

```
List nums = Arrays.asList(1, 2, 3, 4);
nums.forEach(n -> System.out.println(n));
```

Q40. What is the Stream API?

Stream API (Java 8) allows functional-style operations on collections. Key operations: filter(), map(), reduce(), collect(), forEach(). Streams are lazy and do not modify the source.

Q41. What is Optional class?

Optional is a container that may or may not contain a non-null value. Used to avoid NullPointerException. Key methods: isPresent(), get(), orElse(), ifPresent().

Q42. What are default methods in interfaces?

Java 8 allows interfaces to have method implementations using the default keyword. This enables adding new methods to interfaces without breaking existing implementations.

■ SECTION 7: JDBC & Miscellaneous

Q43. What is JDBC?

JDBC (Java Database Connectivity) is an API that enables Java programs to interact with databases. Steps: Load driver → Create connection → Create statement → Execute query → Process results → Close connection.

Q44. What is the difference between Statement and PreparedStatement?

- Statement – Executes static SQL; susceptible to SQL injection; compiled every time.
- PreparedStatement – Precompiled SQL with parameters; prevents SQL injection; better performance.

Q45. What is garbage collection in Java?

Garbage Collection automatically reclaims memory occupied by unreferenced objects. The JVM's GC runs in the background. You can suggest GC with System.gc() but cannot force it.

Q46. What is the difference between deep copy and shallow copy?

- Shallow copy – Copies references to objects, not the objects themselves.
- Deep copy – Creates new copies of all objects recursively. Changes to one do not affect the other.

Q47. What is serialization and deserialization?

- Serialization – Converting an object into a byte stream to save to file or send over network.
- Deserialization – Reconstructing the object from a byte stream. Class must implement Serializable interface.

Q48. What is the difference between HashMap and LinkedHashMap?

- HashMap – Does not maintain insertion order.
- LinkedHashMap – Maintains insertion order using a doubly linked list; slightly slower than HashMap.

Q49. What is a Singleton design pattern?

Singleton ensures only one instance of a class exists throughout the application. Achieved by making the constructor private and providing a static method to get the single instance.

Q50. What is the difference between compile-time and runtime polymorphism?

• Compile-time (Static) polymorphism – Achieved through method overloading; resolved at compile time.

• Runtime (Dynamic) polymorphism – Achieved through method overriding; resolved at runtime using dynamic method dispatch.

■ Interview Tips

✔ Always explain your thought process out loud during interviews.

✔ Practice coding on paper or whiteboard — not just an IDE.

✔ Be clear about the difference between abstract classes and interfaces.

✔ Revise Collections thoroughly — it's the most asked topic for freshers.

✔ Know at least one design pattern well (Singleton, Factory).

✔ Prepare 2–3 projects to discuss confidently.

Best of luck with your interviews! ■